

Lab2 实验报告

1. MoE 前向计算的阶段分析

本实验优化的是一个 DeepSeek-V3 风格的量化 MoE 层前向计算。对每个 token x_t ，计算流程可以概括为一下若干阶段，各个 token 之间呈现极高的可并行度：

```
1  FP32 token x
2  1. Router: sigmoid( W_router x ) -> biased Top-K -> gate normalize
3  2. per-token quantize: x -> xq, scale sx
4      2.1 shared expert: always executed
5      2.2 routed experts: only Top-K experts executed
6  3. output y = x + shared_out + sum( gate_k * routed_out_k )
```

专家 FFN 是 SwiGLU，包含 W_d^e, W_g^e, W_u^e 三个矩阵乘法：

$$FFN_e(x) = W_d^{(e)} \left(SiLU(W_g^{(e)} x) \odot W_u^{(e)} x \right), \quad SiLU(v) = \frac{v}{1 + \exp(-v)}. \quad (34)$$

实验使用 W8A8：权重和激活为 int8，矩阵乘累加到 int32，再乘 scale 回到 fp32。对一个 token，输入量化为

$$s_x = \max_i \frac{|x_i|}{127}, \quad x_q = \text{round}\left(\frac{x}{s_x}\right). \quad (35)$$

一个专家包含 gate/up/down 三个矩阵，计算量近似为：

$$C_{expert} = 3DH. \quad (36)$$

每个 token 总共执行 1 个 shared expert 和 K 个 routed expert，因此专家部分计算量为：

$$C_{token,expert} = (K + 1)3DH. \quad (37)$$

Router 部分是（其中 N 为 token 数， E 为专家数， D 为输入维度）：

$$C_{router} = NED. \quad (38)$$

四个测试场景的主要计算量如下：

Case	N	D	H	E	K	Router MACs	Expert MACs	优化重点
S1	1	256	128	16	4	4,096	491,520	专家并行、低调度开销
S2	1	1024	512	16	4	16,384	7,864,320	大矩阵内部分块并行、专家并行
S3	128	256	128	16	4	524,288	62,914,560	按 expert 聚合后 AMX
S4	1024	512	128	512	2	268,435,456	603,979,776	多专家路由、稀疏 routed 分桶

总体优化策略是：

1. **按 case 专门化执行流**。单 token 和 batch 的并行度完全不同，小矩阵和大矩阵的线程粒度也不同，因此实验中使用 S1-S4 四套内核。当前应对测试的 S1~S4 四个场景，实际应对的不同应用场景（单 token 小矩阵少 expert，单 token 大矩阵少 expert，batch tokens 小矩阵少 expert，batch tokens 大矩阵多 expert。
2. **preprocess 中重排权重**。权重在 timed loop 中反复使用，预打包可以把复杂访存变成连续访存。
3. **AVX-512/VNNI 处理向量级计算**。指数计算，router、量化、requant、SwiGLU、最终累加使用 512-bit SIMD。
4. **AMX 处理 batch int8 GEMM**。在 S3/S4 中，把 token 按 expert 分桶，形成 16 tokens batch + matrix block 的 AMX tile 计算。
5. **控制同步与调度开销**。S1/S2 使用 persistent worker threads，S3/S4 使用 OpenMP 并行区域和任务分桶。

从性能建模角度看，本实验不是单纯“把循环改成 SIMD”即可解决的问题。MoE 的难点在于其计算具有明显的混合特征：Router 是 fp32 dense dot product，专家是 int8 GEMM-like 计算，Top-K 是控制流和比较密集操作，SwiGLU 又回到 fp32 非线性函数。不同阶段的瓶颈并不相同：Router 更依赖 AVX-512 FMA 和 exp 近似，专家投影更依赖 VNNI/AMX 的 int8 dot 吞吐，分桶和合并则更受调度、访存连续性和 cache 行为影响。所以应该综合使用多个优化方式来加速，不仅仅是向量化。

因此优化时采用了“先改变计算组织，再选择指令”的顺序。对 batch case，最重要的是把 reference 中逐 token 的 matrix-vector 组织方式改写成 expert-centric 的小型矩阵乘；对 single-token case，由于无法形成 batch GEMM，只能通过专家并行、矩阵行切分和 persistent worker 来补足线程级并行。换言之，S1/S2 主要解决“没有 batch 时如何并行”，S3/S4 主要解决“有 batch 时如何把稀疏 MoE 组织成 dense tile”。

这个顺序很重要。若直接在 reference 的 token-centric 循环上做局部 SIMD，虽然每个 dot product 会变快，但整体仍然频繁跳转到不同 expert 权重，AMX 也无法利用到足够大的 token 维度。相反，先改变数据流后，很多局部优化会自然成立：同一个 expert 的 token 被放到一起，权重可以连续加载；同一个 token block 的激活被重复用于多个输出行；最终输出按 token 连续累加，又能回到规则的 AVX-512 store。也就是说，算法组织决定了硬件指令是否能真正吃满。应该尽可能将分散的、稀疏的矩阵向量乘法排布为大规模矩阵矩阵乘法。

另一个原则是把 timed loop 中的“可变成本”前移到 preprocess。权重打包、router layout 调整、AMX layout 准备都不依赖输入 token，因此可以在计时外完成。这样 forward 阶段只保留必要的 per-token 计算：router、Top-K、activation quantization 和专家投影。对本 HPC 实验而言，这种思路相当于用一次性内存开销换取多次 forward 的稳定吞吐。

2. 程序分发与四个 Case 的主体流程

student/moe_opt.cpp 负责总分发。preprocess 会同时调用四个 case 的 preprocess（因为 preprocess 不会接收矩阵形状以及 token 的数量，不能具体判断是哪一种 case）；forward 时根据 num_tokens、num_experts、矩阵大小选择内核，判断逻辑如下：

```
1 num_tokens == 1:                                /* 单 token 场景 */
2     D <= 512 && H <= 256 -> S1                  /* 小矩阵规模 */
3     otherwise                                     /* 大矩阵规模 */
4
5 num_tokens > 1:                                  /* bathed token 场景 */
6     E >= 128 -> S4                               /* 多 experts */
7     otherwise -> S3                              /* 少 experts */
```

对应实现中，分发逻辑被写成一个非常小的分类函数。这个函数虽然短，但它决定了后面所有 kernel 可以针对固定形状大胆优化：

```

1  /* Batch 走 S3/S4; single-token 走 S1/S2 */
2  if( num_tokens > 1 )
3      return w.num_experts >= 128 ? ManyExpertBatch : FewExpertBatch;
4
5  /* 小单 token 用轻量 kernel, 大单 token 用更强的专家内并行 */
6  return ( w.d_model <= 512 && w.d_ff <= 256 )
7          ? SmallSingleToken : LargeSingleToken;

```

这几行代码表达的不是简单分支，而是把不同规模下的瓶颈提前固化：batch 场景优先按 token/expert bucket 并行，多专家场景额外重视 router/top-k 的讨论；单 token 场景则根据矩阵大小决定是否需要更细的专家内部的大矩阵计算并行。

这种分发的核心原因是：**并行度来源不同**：

- $N = 1$ 时没有 token 级并行，只能从专家和矩阵内部找并行度，而且需要压缩多线程通信开销；
- $N > 1$ 时 token 级并行天然存在，但 routed experts 是稀疏访问，需要先组织数据；
- E 很大时 router 和 top-k 不再是小开销，必须专门优化，将各个 token 的 router 计算组织为矩阵计算；
- 矩阵大时，单个专家本身就足够重，可以进一步拆分 gate/up/down 的多线程计算。

2.1 S1 case: 单 token、小矩阵场景

场景：

```

1  N = 1, D = 256, H = 128, E = 16, K = 4

```

主体流程：

```

1 preprocess:
2     pack router 4 x 16
3     pack gate/up/down for VNNI
4     pack shared expert
5
6 forward one token:
7     1. AVX-512 router dot + sigmoid
8     2. SIMD top-k
9     3. AVX-512 quantize token
10    4. 启动/复用 routed expert worker
11    5. 主线程计算 shared expert
12    6. worker 计算 K 个 routed experts          /* 专家级并行 */
13    7. AVX-512 合并 x + shared + routed

```

S1 的专家计算量约 **491K** int8 MACs，router 只有 **4K** fp32 MACs。瓶颈显然在专家 FFN。由于只有一个 token，OpenMP 按 token 并行没有意义；如果每次 forward 都创建线程，线程创建开销也会超过计算收益。因此 S1 使用 persistent worker threads：线程在程序生命周期内存在，通过 atomic epoch 接收任务。这样可以显著降低线程通信的开销，将线程首次创建的开销摊消到若干次 forward 的过程中。

并行流如下：

```

1 main thread:
2     router / top-k / quantize
3     publish routed expert tasks
4     compute shared expert
5     wait workers
6     combine
7
8 worker 0 ... K-1:
9     wait epoch
10    compute one routed expert
11    publish done_epoch

```

- shared expert 和 K 个 routed experts 互相独立
- 每个专家都是一个一定规模的 FFN，任务粒度足够覆盖 atomic 同步开销
- persistent threads 避免反复创建/销毁线程，而如果使用 openmp 则需要每次进入并行区域时创建线程，这个开销在该场景下不可忽视
- 对单 token 场景，专家级并行是最自然的并行度

这里的一个关键取舍是“并行粒度不能太细”。S1 每个 routed expert 的矩阵规模为 $D = 256, H = 128$ ，单个 gate/up/down 投影虽然可以继续按行拆分，但拆分后每个任务的计算量会很小，线程同步、任务发布、cache 竞争的成本会快速上升。因此 S1 选择以“一个 expert”为基本任务粒度：每个 worker 完整计算一个 routed expert，主线程同时计算 shared expert。这样每个线程拥有较长的连续计算段，权重访问也相对集中。我试过更细粒度的并行，但是这个时候的通信开销占比会很高。

从流水角度看，S1 的热路径主要是 packed VNNI dot product、SwiGLU 和 down projection。由于输入只有一个 token， xq 很小，可以长期停留在 L1/L2 cache；主要带宽压力来自专家权重。因此权重打包的收益很明显：它把原始按矩阵行存储的权重变成 VNNI 友好的连续 block，减少内层循环中的地址计算和跨 cache line 跳转。

S1 还有一个隐含的延迟优化目标：单 token 场景更关注单次 forward latency，而不是 batch throughput. persistent worker 的好处不仅是减少线程创建成本，还能让线程在固定位置等待下一轮任务，避免 OpenMP runtime 在很小任务上引入不可控调度开销。主线程计算 shared expert 与 worker 计算 routed expert 同时进行，相当于把 MoE 中天然独立的 $K+1$ 个专家分支铺开执行；最后合并阶段只做连续向量加法，延迟较低且容易预测。

2.2 S2 case: 单 token、大矩阵场景

场景：

```
1 N = 1, D = 1024, H = 512, E = 16, K = 4
```

主体流程与 S1 类似，但矩阵规模大得多。专家部分计算量约 **7.86M** int8 MACs，是 S1 的 16 倍。因此 S2 不只并行不同 routed experts，还进一步把大专家内部的 gate/up/down 投影拆成更细粒度任务。

S2 的执行流：

```
1 1. router/top-k/quantize
2 2. routed expert workers 处理不同专家
3 3. 每个专家内部：
4     gate/up projection 可拆 2 range
5     hidden requant
6     down projection 可拆 range
7 4. shared expert 使用同样的 hybrid kernel
8 5. AVX-512 合并输出
```

S2 中对于单个 expert 存在 gate/up helper 和 down helper。这样设计是因为：

- 大矩阵下单个专家计算量很大，只给一个线程处理会留下大量未利用核心（因为上限是 16 个线程，S1 场景只使用 5 个线程，未充分利用计算资源）；
- gate 和 up 对同一个 x_q 做两个独立矩阵乘，访存模式类似，可以合并/交错执行，提高输入向量的复用；
- down projection 输出维度 $D = 1024$ ，按输出行切分无写冲突；
- 专家间并行和专家内并行结合，可以在 $N = 1$ 时制造足够线程级并行。

计算密度上，大矩阵相比小矩阵更适合细分任务：每个任务有更多 int8 dot product，可以摊薄任务分发和同步成本。S1 中如果过度拆分，线程同步会吞掉收益；S2 中拆分更划算。

S2 和 S1 的根本区别是“单个专家是否足够大”。在 S2 中，一个专家的计算量为：

$$3DH = 3 \times 1024 \times 512 = 1,572,864 \text{ MACs.} \quad (39)$$

这已经足够让一个专家内部的投影成为长时间运行的计算段。如果仍然只让一个线程负责一个 expert，那么最多只有 shared + 4 routed 共 5 个并行任务，无法充分利用 16 核配置。S2 的 hybrid 方案通过 gate/up/down helper 把一个 expert 的内部工作也拆出来，形成更多并行任务。因为 D 和 H 都很大，按 range 切分后每块仍有足够高的计算密度，线程调度成本不会成为主导。

另一个重要考虑是内存层次。S2 的权重规模远大于 S1，单次 forward 会触及更多 cache line. gate 和 up 共享同一个 x_q ，如果把两者交错计算，就能让 x_q 的 load 在多个 dot product 中复用；down projection 则按输出行切分，各线程写不同输出位置，避免写冲突。这种划分既提高了并行度，也维持了较好的访存连续性。

S2 中专家内并行需要格外控制同步边界。gate/up 投影、SwiGLU、requant、down 投影之间存在数据依赖，不能简单全部乱序并行；但在每个阶段内部，按 hidden row 或 output row 切分是安全的。实现上选择 helper 线程协助较重的 range，而不是把每一小段 dot 都做成独立任务，是为了让每个任务拥有足够长的计算区间。这样线程拿到任务后可以连续执行大量 VNNI 指令，避免任务队列或 atomic 状态频繁成为瓶颈。关键就是不要让通信开销成为主导因素。

2.3 S3 case: batch tokens、少专家场景

场景：

1 $N = 128, D = 256, H = 128, E = 16, K = 4$

主体流程：

```
1 preprocess:
2     pack router 4 x 16
3     pack shared/routed weights into AMX layout
4
5 forward:
6     parallel for token:
7         router score
8         top-k
9         gate_sum
10        quantize x -> xq
11
12    shared_expert_amx_fused:
13        AMX gate/up for all tokens
14        SIMD SwiGLU
15        SIMD requant
16        AMX down
17        y = x + shared_out
18
19    process_routed_experts_bucketed:
20        build expert buckets from top-k
21        split each bucket into 16-token tasks
22        OpenMP execute AMX gate/up -> requant -> AMX down
23        vectorized add routed_out into y
```

S3 的平均每 expert assignment tokens 数为：

$$\frac{NK}{E} = \frac{128 \times 4}{16} = 32. \quad (40)$$

这非常适合 AMX：每个 expert 平均有两个完整的 16-token tile。若仍按 token 逐个计算，每个 token 只做 matrix-vector，无法发挥 AMX matrix-matrix 的吞吐。根据 lab2 文档中的 hint 换成 expert 视角后：

```

1 token-centric:
2     for token:
3         for selected expert:
4             matrix-vector
5
6 expert-centric:
7     for expert:
8         collect tokens selecting this expert
9         process 16 tokens as one matrix-matrix tile

```

数据组织如下：

```

1 top-k by token:
2     token 0 -> e3, e7, e1, e9
3     token 1 -> e7, e2, e3, e5
4     token 2 -> e3, e4, e7, e1
5
6 bucket by expert:
7     expert 3 -> token 0(slot0), token 1(slot2), token 2(slot0), ...
8     expert 7 -> token 0(slot1), token 1(slot0), token 2(slot2), ...

```

slot 保存该 expert 是 token 的第几个 top-k 结果，用于写回：

```

1 base = token_id * MAX_TOP_K + slot
2 g_routed_h[ base ], g_routed_hq[ base ], g_routed_out[ base ]

```

代码中分桶逻辑直接对应上面的数据结构。第一段把 token-centric 的 top-k 结果转为 expert-centric bucket，关键语句如下：

```

1 int e = g_batch_topk[ t * MAX_TOP_K + k ];          /* token t 的第 k 个 expert */
2
3 int idx = g_routed_bucket_count[ e ]++;             /* 放入 expert e 的 bucket */
4 g_routed_bucket_token[ e * max_assign + idx ] = t; /* 记录原始 token id */
5 g_routed_bucket_slot[ e * max_assign + idx ] = k;  /* 记录 top-k slot */

```

第二段把每个 expert 的 bucket 按 AMX 的 token tile 大小切成任务，也只需要记录 expert、起点和长度：

```
1 g_routed_task_expert[ num_tasks ] = e;      /* task 属于哪个 expert */
2 g_routed_task_start[ num_tasks ] = done;    /* bucket 内起点 */
3 g_routed_task_count[ num_tasks ] = min( 16, count - done ); /* token 数 */
```

这几行体现了 S3/S4 的核心组织方式：先用很轻量的索引数组描述路由关系，再把真正重的 FFN 计算组织成 AMX 可以消费的 16-token task。不满 16 的 task 在执行时补 0，保持同一套 AMX kernel。

S3 的并行性来自三个角度：

1. 前处理按 token 并行；
2. shared expert 按 token block / dim block 并行；
3. routed experts 按 bucket task 并行，默认 `schedule( dynamic, 1 )` 以适应路由不均衡。

S3 中 Router 的计算量相对于专家计算量较小，因此没有像 S4 那样专门做 batch router tile。E = 16 时，一个 token 的所有 router score 可以放进一个 `__m512` 中处理，Top-K 也可以在一个向量内完成。此时把工程复杂度投入到 router 上收益有限，主要收益来自把专家计算转化成 AMX tile。

shared expert 在 S3 中尤其适合先做，因为它对所有 token 都执行，不依赖 Top-K 分桶结果。前处理阶段得到所有 token 的 `xq/s_x/correction` 后，shared expert 可以直接按 16 tokens 一组进入 AMX。它完成后写出 `y = x + shared_out`，routed expert 阶段只需要增量累加。这种“shared first”的布局减少了输出缓冲数量，也避免 shared 和 routed 同时写 `y` 的同步问题。

routed expert 的分桶则解决了 MoE 稀疏性与 AMX dense tile 之间的矛盾。虽然每个 token 只选 4 个专家，但从 expert 角度看，E = 16 且 N = 128 使得每个 expert 平均收到 32 个 token，足以形成稳定的 AMX batch。动态调度的必要性来自实际 router 分布不会完全均匀：某些 expert 可能被更多 token 选中，某些较少。`schedule( dynamic, 1 )` 让长 bucket 不会拖住固定线程，各线程负载保持均衡。

S3 的执行流还有意把“按 token 的轻工作”和“按 expert 的重工作”分开。Router、Top-K、输入量化都只依赖单个 token，适合直接 `parallel for token`；专家 FFN 则依赖同一个 expert 的多 token 聚合，适合 bucket 后再执行。这样每个阶段的并行维度都与数据依赖一致，避免了在专家计算过程中频繁查询 Top-K 或做复杂同步。前半段产生 compact metadata，后半段消耗 metadata 做高吞吐计算，是 S3 能稳定加速的关键。

2.4 S4 case: batch、多专家场景

场景：

```
1 N = 1024, D = 512, H = 128, E = 512, K = 2
```

S4 的特点是 router 和 top-k 成本显著增大：

$$NED = 1024 \times 512 \times 512 = 268,435,456. \quad (41)$$

因此 S4 和 S3 最大区别是专门优化 batch router 和 top-2。

主体流程：

```
1 preprocess:
2     pack router 4 x 16
3     pack all expert/shared weights into AMX layout
4
5 forward:
6     compute_router_scores_batch:
7         for expert groups of 4:
8             router_tile_kernel_4x4 over tokens
9
10    parallel for token:
11        select_topk_2_simd
12        gate_sum
13        quantize x
14
15    shared_expert_amx_fused:
16        y = x + shared_out
17
18    process_routed_experts_bucketed:
19        bucket by expert
20        split into 16-token AMX tasks
21        AMX gate/up -> requant -> AMX down
22        vectorized add routed_out into y
```

S4 中 routed expert 的平均 assignment 数为：

$$\frac{NK}{E} = \frac{1024 \times 2}{512} = 4. \quad (42)$$

这意味着 routed bucket 比 S3 稀疏得多；但是总任务数较大，且我们当前实现统一为 AMX 路径，尾块不足 16 token 时补零。这样可以简化执行流，让所有 routed expert 使用相同内核。代价是小 bucket AMX 利用率不满，但好处是所有专家计算路径一致，也可以保证 AMX kernel 吞吐高，即使有补零也可能优于复杂 fallback。

S4 的 router 使用固定 4 experts x 4 tokens tile:

```
1 current expert group: e, e+1, e+2, e+3
2 current token group : t, t+1, t+2, t+3
3
4 compute 16 scores:
5
6           e+0  e+1  e+2  e+3
7 token t      .    .    .    .
8 token t+1    .    .    .    .
9 token t+2    .    .    .    .
10 token t+3   .    .    .    .
```

这样做的原因包括：

- 4 行 expert 正好匹配 pack_router_4x16 的 4-row layout；
- 4 tokens 使权重 $w_0 \dots w_3$ 在同一个 cb 上被复用 4 次；
- 16 个 accumulator 对应 4x4 dot products，能利用 AVX-512 的 32 个 ZMM 寄存器，但不会过度溢出；
- $D = 512$ ，每个 dot 有 32 个 16-float chunks，循环体足够长，可以隐藏 FMA 和 load 延迟。

S4 中 Top-K 的特点也不同。 $E = 512$ ， $K = 2$ 意味着每个 token 需要在 512 个 biased score 中选出两个最大值。如果直接使用通用 Top-K，每轮扫描全部 experts， $K = 2$ 就要至少两遍；而专用 top-2 SIMD 可以一遍扫描维护局部 top1/top2，再归并 32 个候选。由于 Top-K 位于每个 token 的必经路径上，且 $N = 1024$ ，这个优化直接影响端到端性能。

S4 routed expert 的平均 bucket 只有 4 个 token，看似不适合 AMX 的 16-token tile。但 S4 的 routed 总 assignment 数为 2048，expert 数多、任务多，统一 AMX 路径可以让代码结构更简单，并减少不同 fallback 路径之间的分支和预打包开销。实践中，小 bucket 补零会降低单个 tile 利用率，但 AMX 的 int8 TMUL 吞吐很高，且统一路径减少了控制复杂度。这里体现的是“算术利用率”和“执行流复杂度”之间的 trade-off.

与 S3 相比，S4 的瓶颈更分散：router 本身已经是数亿级 fp32 MAC，Top-2 又要扫描 512 个 expert；routed expert 虽然总计算量也大，但每个 expert 的 token 数较少，局部 tile 利用率不如 S3。于是 S4 的优化重点不能只放在 AMX 上，而必须同时压缩 router 和 Top-K 的成本。4x4 router tile、专用 top-2 SIMD、统一 AMX routed path 这三者分别对应 dense FP32、比较选择、稀疏 int8 三类热点。

3. 权重打包与数据布局优化

3.1 Router 4x16 打包

Router 原始布局：

```
1 w_router[ e ] [ d ]
2
3 expert 0: d0 d1 ... d15 | d16 d17 ...
4 expert 1: d0 d1 ... d15 | d16 d17 ...
5 expert 2: d0 d1 ... d15 | d16 d17 ...
6 expert 3: d0 d1 ... d15 | d16 d17 ...
```

打包后按 4 experts x 16 dims 分块：

```
1 block( e = 0 ... 3, d = 0 ... 15 ):
2     expert 0, dim 0 ... 15
3     expert 1, dim 0 ... 15
4     expert 2, dim 0 ... 15
5     expert 3, dim 0 ... 15
6
7 block( e = 0 ... 3, d = 16 ... 31 ):
8     expert 0, dim 16 ... 31
9     ...
```

这样在计算 4 个 expert 的 dot product 时，可以连续加载 4 条 512-bit 权重向量：

```

1 w0 = load( row0 + coff );      /* expert e+0 的 16 维权重块 */
2 w1 = load( row1 + coff );      /* expert e+1 的 16 维权重块 */
3 w2 = load( row2 + coff );      /* expert e+2 的 16 维权重块 */
4 w3 = load( row3 + coff );      /* expert e+3 的 16 维权重块 */

```

对 S4 的 4x4 router tile, $w_0 \dots w_3$ 被 4 个 token 复用：

```

1 load weights once
2 load x0 -> 4 FMA
3 load x1 -> 4 FMA
4 load x2 -> 4 FMA
5 load x3 -> 4 FMA

```

这提高了权重加载的计算密度。每个 cb 加载 4 个权重向量和 4 个 token 向量，共 8 次 64B load，完成 16 个 FMA 向量，也就是 $16 * 16 = 256$ lane-level FMA。权重复用是该 kernel 的核心优化方法。从 cache 行为看，router 权重在 S4 中会被大量 token 重复使用。如果仍保持按 expert 行独立存储，计算 4 个 expert 时虽然也能连续读每一行，但难以自然表达“同一列块上同时处理 4 行”。4x16 打包把四个 expert 的同一维度块放到相邻位置，使内层循环的地址推进非常规则。规则地址不仅减少指针计算，也让硬件预取更容易命中。对 router 这种 FP32 计算来说，FMA 吞吐和 load 带宽都重要，打包的意义就是让两者都保持稳定。

3.2 VNNI u8-bias 打包

AVX-512 VNNI 的 `_mm512_dpbusd_epi32` 适合 unsigned int8 与 signed int8 的点积。原始权重是 signed int8，因此我们把权重做：

$$w_u = w_s + 128. \quad (43)$$

代码中用 `^ 0x80` 完成符号位翻转，等价于把 signed byte 映射到 unsigned-biased 表示。点积变成：

```

1 packed_w = ( int8_t )( ( uint8_t )w_signed ^ 0x80u );      /* signed -> u8-
   biased */
2 signed_dot = acc_u8s8 - 128 * sum_xq;                      /* 减去 bias
   correction */
3 \sum_i (w_i + 128)x_i
4 = \sum_i w_i x_i + 128 \sum_i x_i.

```

因此计算后需要修正：

$$\sum_i w_i x_i = acc_{u8s8} - 128 \sum_i x_i. \quad (44)$$

这就是 quantize_token 同时返回 sum_xq 的原因：

```

1 g_batch_x_correction[ t ] = sum_xq * 128;                /* 同一个 token 的所有输出行复用
   */

```

hidden requant 后同理：

```

1 g_routed_h_correction[ base ] = sum_hq * 128;           /* down projection 的
   correction */

```

这个技巧用一次简单的加法修正，换取了 VNNI/AMX 对 int8 dot 的高吞吐。

需要注意的是，correction 只和 activation 的量化值之和有关，与具体输出行无关。因此对同一个 token 的 gate/up 投影，sum_xq * 128 可以被所有 hidden rows 复用；对 down projection，sum_hq * 128 可以被所有 output rows 复用。这样 correction 的代价被摊薄到整行矩阵乘上，而不是在每个乘法元素上付出额外成本。这个性质也是为什么量化阶段要同时保存 sum_xq，requant 阶段要同时保存 sum_hq。

3.3 AMX Kx16 VNNI 打包

AMX tile 使用 16 output rows x 64 K 的逻辑权重块。由于 _tile_dpbsud 按 VNNI 的 4-byte group 消费 K 维数据，权重实际打包时不是按 logical row 连续存完整 64 个 K，而是先按 kg 分组；每个 kg 表示连续 4 个 K 值：

```

1  /* logical weight block: */
2      16 output rows x 64 input cols
3
4  /* packed order: */
5      for kg in 0 ... 15:          /* 每个 kg 覆盖 4 个 K */
6          for row in 0 ... 15:      /* 遍历 16 个输出 row */
7              store weight[ row ][ kg * 4 + 0 ... 3 ]

```

示意：

```

1  logical W rows:
2
3  row0: k0 k1 k2 k3 | k4 k5 k6 k7 | ...
4  row1: k0 k1 k2 k3 | k4 k5 k6 k7 | ...
5  ...
6  row15
7
8  packed order:
9
10 kg0: row0 k0..k3 | row1 k0..k3 | ... | row15 k0..k3
11 kg1: row0 k4..k7 | row1 k4..k7 | ... | row15 k4..k7
12 ...
13 kg15: row0 k60..k63 | row1 k60..k63 | ... | row15 k60..k63

```

这样 `_tile_loadd` 可以连续读取权重 tile，避免运行时 gather 或跨行跳跃访问。需要注意的是，这里的 AMX tile row 对应的是 `kg`，不是 logical weight row；一个 tile row 中包含 16 个输出 row 各自的 4 个 K 值，即 $16 * 4 = 64$ 字节。权重不随 iteration 改变，把重排放进 preprocess 是典型的以空间换时间。

AMX 对数据布局比普通 AVX-512 更敏感。AVX-512 中即使布局不理想，也可以通过多次 load、shuffle 或手工 gather 勉强拼出向量；AMX tile 则要求 tile memory layout 与 `_tile_dpbsud` 的解释方式匹配，否则每个 K block 都要在运行时重排，代价会非常高。预打包后，forward 阶段的内层循环基本只剩下 `_tile_loadd` 和 `_tile_dpbsud`，这让 AMX 的 TMUL 单元能够持续工作，而不是等待软件整理数据。

4. 其他函数级优化

4.1 快速 exp 近似优化

Router sigmoid 和 SiLU 都需要 `exp` 指数计算：

$$\sigma(x) = \frac{1}{1 + \exp(-x)}, \quad \text{SiLU}(x) = x\sigma(x). \quad (45)$$

标准 `expf` 是标量库函数，延迟高、难向量化。我们实现了两个 AVX-512 近似：

- `fast_exp_ps` : 用于 SiLU，三次多项式，速度更快；
- `router_exp_ps` : 用于 router sigmoid，五次多项式，精度更高。

数学推导如下。先利用：

$$\exp(x) = 2^{x \log_2 e}. \quad (46)$$

令：

$$y = x \log_2 e, \quad n = \lfloor y \rfloor, \quad f = y - n, \quad f \in [0, 1). \quad (47)$$

则：

$$\exp(x) = 2^{n+f} = 2^n 2^f. \quad (48)$$

AVX-512 中 2^n 可以通过 `_mm512_scalef_ps` 高效实现。因此问题变成近似 2^f 。在 $f \in [0, 1)$ 上用多项式：

$$2^f \approx 1 + c_1 f + c_2 f^2 + c_3 f^3 \quad (49)$$

或更高阶：

$$2^f \approx c_0 + c_1 f + c_2 f^2 + c_3 f^3 + c_4 f^4 + c_5 f^5. \quad (50)$$

代码使用 Horner 形式：

```

1  n = floor( x * log2e ); // 拆出 2^n
2  f = x * log2e - n;      // 剩余小区间 f in [0, 1)
3  p = f * c3 + c2;        // Horner 多项式
4  p = f * p + c1;
5  p = f * p + 1;
6  exp_x = scalef( p, n ); // p * 2^n

```

Horner 形式（秦九韶算法）减少乘法次数，且可以用 FMA：

$$p = f(c_1 + f(c_2 + fc_3)) + 1. \quad (51)$$

计算密度角度分析一下：

- 一次处理 16 个 fp32
- 多项式只使用 FMA/add/mul，避免标量函数调用
- 指令流无分支，适合软件流水化
- 对 router 使用更高阶近似，降低 top-k 误选风险
- 对 SiLU 使用低阶近似，因为最终有 int8 requant 可以容忍小误差

这里还存在一个精度与吞吐的折中。Router 的 sigmoid 会影响 Top-K，一旦近似误差导致 expert 排名交换，后续 routed expert 将完全不同，输出误差会明显放大。因此 router_exp_ps 使用更高阶多项式，牺牲少量 FMA 换取更稳定的排序。SwiGLU 中的 exp 只影响 hidden activation 的连续值，后续又会进入 per-token hidden requant，局部误差通常被 RMSE 容忍范围吸收，因此 fast_exp_ps 可以使用更低阶近似。

Horner 形式的另一个好处是依赖链短且指令类型规则。虽然多项式本身存在串行依赖，但每条 AVX-512 指令处理 16 个元素，且不同 token/expert 的 exp 调用处通常还有其他 load、mul、add 可以与之交错。相比调用 expf，它避免了函数调用、循环分支、查表和标量/向量之间的数据搬运。

从数学误差角度看，exp 近似的输入会先被 clamp 到有限范围，避免指数过大产生 Inf，也避免过小产生完全下溢后破坏 sigmoid 的单调性。对 sigmoid 而言，真正敏感的区域在 0 附近，因为此时斜率最大；当输入绝对值很大时，sigmoid 已经接近 0 或 1，近似误差对结果影响反而较小。因此用多项式逼近 2^f 时，更关心中间区间的相对误差和单调性，而不是追求与 libm expf 完全逐 bit 一致，毕竟这是没有必要的。

4.2 quantize_token 向量化优化

量化需要两步：

$$s_x = \max_i \frac{|x_i|}{127}, \quad x_q = \text{round}\left(\frac{x}{s_x}\right). \quad (52)$$

实现上使用 AVX-512 做向量化：

```
1 pass 1:
2     load 16 floats
3     abs via sign mask
4     vector max
5     reduce max
6
7 pass 2:
8     load 16 floats
9     multiply inv_sx
10    convert fp32 -> int32
11    narrow int32 -> int8
12    store 16 int8
13    accumulate sum_xq
```

对应到代码，最关键的是这几步：

```
1 x_amax = reduce_max( abs( x ) );    /* pass 1: 找整行最大绝对值 */
2 s_x = x_amax / 127.0f;              /* fp32 -> int8 的 scale */
3 xq = round( x / s_x );              /* pass 2: 量化 */
4 sum_xq += xq;                      /* 顺便为 correction 求和 */
```

为什么分两 pass？因为 scale 依赖整行最大绝对值，必须先知道 amax。第二 pass 做量化的同时计算 sum_xq，为 u8-bias correction 服务。这样避免额外第三 pass。

寄存器容量角度，单次 16 floats 正好填满一个 ZMM。d_model 保证为 64 倍数，因此没有尾部 mask，循环结构非常干净。

量化阶段虽然计算量不如专家矩阵乘大，但它位于每个 token 的必经路径上，并且会影响所有后续专家计算。因此它需要同时满足两个主要目标：一是快，二是把后续需要的元信息一并产出。sum_xq 的累加就是一个例子。如果不在量化时顺手求和，后面做 VNNI/AMX correction 时还要再扫描一次 xq，这会引入额外的内存流量。当前实现把 max、convert、sum 三类操作压缩到两次线性扫描中，访存模式完全连续，对 cache 和硬件预取器都比较友好。

这里没有采用逐元素标量 clamp/cast，而是让向量转换指令直接完成 fp32 -> int32 -> int8 的窄化。由于 scale 由 $\text{amax}/127$ 给出，理论上量化结果落在 $[-127, 127]$ ，不会触及 int8 的极端溢出路径。这样可以减少额外的 min/max 饱和操作。更重要的是，量化输出 xq 后续会以 64 字节块喂给 VNNI/AMX，因此按连续 16 字节写出能够保持紧凑布局，避免后续再做 format conversion.

4.3 ffn_gate_up_packed 向量化优化

gate/up 是两个矩阵对同一个输入向量 xq 的投影：

$$v_g = W_g x_q, \quad v_u = W_u x_q. \quad (53)$$

实现中一次处理 4 个 hidden rows，d_model 每 64 列一块。使用 _mm512_dpbusd_epi32：

```
1 load xq[ 64 ]
2 load gate row0 block -> dpbusd
3 load up row0 block -> dpbusd
4 load gate row1 block -> dpbusd
5 load up row1 block -> dpbusd
6 ...
```

这种交错顺序有两个目的：

1. gate/up 共享同一个 xq load，提高输入复用，增大计算强度。
2. load 与 dot 交错，给乱序执行器更多机会隐藏 load latency. 做好流水化。

完成 int32 dot 后：

$$v_g = (acc_g - correction)s_x s_g, \quad v_u = (acc_u - correction)s_x s_u. \quad (54)$$

随后计算：

$$h = SiLU(v_g) \odot v_u. \quad (55)$$

这里 16 个 hidden 元素一起走 AVX-512，使用快速 exp 近似。计算 h 时顺便求 $\max(\text{abs}(h))$ ，为后续 requant 提供 scale 数值，避免后面再次遍历 token 向量的全部维度。

`ffn_gate_up_packed` 的设计重点是提高输入向量和指令流水的利用率。`gate/up` 两个矩阵共享同一个 `xq`，因此每读取一次 `xq[ 64 ]`，就尽量完成多个 `gate/up row` 的 dot product。

`_mm512_dpbusd_epi32` 一次将 64 个 `int8` 输入与 `packed` 权重做点积，并累加到 16 个 `int32 lane` 中。多个 `row` 的 `accumulator` 彼此独立，可以减少单一 `accumulator` 依赖链过长的问题。这里没有把 `hidden rows` 一次展开得过大，是因为 `ZMM` 寄存器数量有限。每增加一组 `row`，就需要更多 `gate/up accumulator`、临时权重向量和输入向量；超过一定规模后编译器可能把寄存器 `spill` 到栈上，反而增加内存访问。4 行 `block` 是在 `VNNI` 吞吐、寄存器压力和代码复杂度之间的折中。

`gate/up` 合并计算还减少了中间结果的生命周期。如果先完整算完 `gate`，再完整算 `up`，必须保存一整段 `gate` 结果，之后再读回来与 `up` 做 `SwiGLU`；交错计算则可以在较小的 `hidden block` 内同时得到 `gate` 和 `up`，马上做 `scale`、`SiLU` 和乘法。这缩短了中间 `fp32` 数据在内存中的停留时间，也降低了 `cache` 压力。对 `S1/S2` 这种 `matrix-vector` 路径来说，这类局部复用尤其重要，因为它不能像 `AMX batch` 那样依靠 `token` 维度放大算术强度。

4.4 ffn_requantize 向量化优化

hidden 量化：

$$s_h = \max_i \frac{|h_i|}{127}, \quad h_q = \text{round}\left(\frac{h}{s_h}\right). \quad (56)$$

与 `token quantize` 类似，使用 `AVX-512` 做 `max`、`scale`、`convert`、`narrow`，并同时计算：

$$\sum_i h_{q,i}. \quad (57)$$

后续 `down projection` 使用 `u8-biased weight`，因此 `correction` 是：

$$128 \sum_i h_{q,i}. \quad (58)$$

`Requantize` 的隐藏成本在于它连接了 `FP32` 非线性和 `int8 down projection`。为了使用 `int8 down GEMM`，必须把 `h` 再量化成 `hq`，而 `scale` 又依赖整段 `hidden` 的 `amax`。因此该函数和 `quantize_token` 类似，也采用“先 `max` 后 `convert`”的两阶段结构。不同之处在于 `hidden` 的数值分布由 `SwiGLU` 产生，可能比输入 `token` 更尖锐，所以 `amax` 的准确性直接影响 `down projection` 的整体缩放。

同时保存 `sum_hq` 是为了让 `down projection` 继续使用 `u8-biased packed` 权重。也就是说，`requant` 不只是格式转换，还承担了为下一个 `int8 dot kernel` 准备 `scale` 和 `correction` 的作用。

Requantize 的位置也体现了 W8A8 的整体取舍：非线性 SwiGLU 保留在 FP32 中计算，保证激活函数的数值稳定；进入 down projection 前再压回 int8，使最大的矩阵乘仍然走 VNNI/AMX。这样做比全程 FP32 便宜得多，也比强行用低精度近似非线性更安全。代价是多一次量化扫描，但 hidden 维度 H 相对矩阵乘的 DH 计算量要小得多，这个扫描通常可以被后续 int8 GEMM 的收益覆盖。

4.5 AMX shared expert 优化

S3/S4 中 shared expert 对所有 token 都执行，天然就是 dense batch GEMM。以 N = 128/1024 为例，shared expert 不受 top-k 稀疏影响，因此最适合 AMX（《如果你追求性能的极致提升，请一定要使用 AMX》）。

shared gate/up 阶段：

```
1  for token block tb in 16:                                /* 外层循环是 token */
2      for hidden block f in 16:                             /* 内层循环是 hidden rows 维 */
3          tile0 = 0                                           /* gate accumulator */
4          tile1 = 0                                           /* up accumulator */
5          for D block in 64:                                  /* K 维 */
6              tile2 <- xq[ 16 tokens x 64 dims ]
7              tile3 <- sh_gate[ 16 rows x 64 dims packed ]
8              tile4 <- sh_up [ 16 rows x 64 dims packed ]
9              tile0 += tile2 dot tile3
10             tile1 += tile2 dot tile4
11         store acc
12         SIMD scale + correction + SiLU
```

实际 AMX kernel 的关键动作可以压缩成三行：

```
1  _tile_loadadd( 2, x_tile, stride );                        /* 16 tokens x 64 dims */
2  _tile_loadadd( 3, w_tile, stride );                        /* 16 rows x 64 dims */
3  _tile_dpbsud( 0, 2, 3 );                                  /* tile0 += tile2 dot tile3 */
```

down 阶段：

```

1 for token block tb in 16:                                /* 外层循环是 token 维 */
2     for output dim block d in 16:                        /* 内层循环是输出维 */
3         tile0 = 0
4         for H block in 64:                                /* K 维 */
5             tile2 <- hq[ 16 tokens x 64 hidden ]
6             tile3 <- sh_down[ 16 output rows x 64 hidden packed ]
7             tile0 += tile2 dot tile3
8         y = x + dequant( tile0 )

```

AMX 的优势在于 tile 内部复用：

- 一个 xq tile 同时服务 16 output rows
- 一个 weight tile 同时服务 16 tokens
- int8 dot 在 TMUL 单元中高吞吐执行
- 与 AVX-512 相比，AMX 减少了显式内层 dot 指令数量

从计算密度看，AMX 的收益来自把 16 个 token 与 16 个输出行组成一个小矩阵乘。若使用 AVX-512 matrix-vector，每个 token 都要单独读取同一块权重；AMX 中同一个 weight tile 被 16 个 token 共享，同一个 activation tile 也被 16 个输出行共享。这样每字节权重/激活对应的乘加次数明显增加，算术强度更高。

AMX 也改变了流水化方式。AVX-512 需要在 C++ 循环中显式发出大量 dot 指令；AMX 的 `_tile_dpbsud` 一条指令驱动 TMUL 单元完成一个 tile 点积，程序只需要在 K 维 block 上循环加载 tile 并累加。为了避免单个 tile accumulator 的长延迟暴露，shared gate/up 同时维护 gate 和 up 两个 accumulator，使得 load、TMUL 和 store 之间有更多独立操作可以交错。

shared expert 先于 routed expert 执行，还有输出组织上的好处。shared expert 对每个 token 都必然存在，因此可以直接初始化最终输出 $y = x + \text{shared_out}$ ；routed expert 之后只做增量累加。若 shared 和 routed 同时写 y，就需要额外同步或临时 buffer；当前顺序把写冲突变成了阶段间顺序，OpenMP barrier 后再进入下一阶段即可。这个设计牺牲了一点阶段间重叠的可能性，但换来了更简单、更确定的内存写入方式。

4.6 Routed expert bucketing 分块优化

MoE 的 routed experts 是稀疏的。朴素 token-centric 流程会导致：

```

1 for token:
2     for selected expert:
3         read expert weight
4         matrix-vector

```

上面的 token 视角做的是矩阵向量乘法，但是 AMX 适合于做矩阵矩阵乘法，这对 AMX 很不友好。我们的优化是转换视角：

```

1 for token:                                /* token -> expert */
2     append token to bucket[ selected expert ]
3 for expert:                                /* expert -> token */
4     process bucket tokens in chunks of 16

```

bucket 数组：

```

1 g_routed_bucket_count[ e ]                /* 专家 e 需要处理的 tokens 总
   数 */
2 g_routed_bucket_token[ e ][ i ] = token id /* 专家 e 处理的第 i 个 token
   在全局的编号 */
3 g_routed_bucket_slot[ e ][ i ] = top-k slot id /* 专家 e 在其处理的第 i 个
   token 的 topk 中的编号 */

```

每个 task：

```

1 ( expert e, bucket start, count <= 16 )

```

执行：

```

1 routed_gate_up_amx_chunk
2 routed_requantize_chunk
3 routed_down_amx_chunk

```

routed_gate_up_amx_chunk 把同一 expert 的最多 16 个 token 拼成 x_tile，不足 16 的位置补 0。这样可复用同一个 expert 的 gate/up 权重 tile。routed_down_amx_chunk 会把输出直接乘以 gate：

$$out = (acc - correction)(s_h s_{down})g_{t,e}. \quad (59)$$

最后只需要按 (token, slot) 把 g_routed_out 加回 y。

调度策略是 `schedule( dynamic, 1 )`：默认，适合 bucket 大小不均衡。

分桶的代价主要是额外的索引写入和后续按 `(token, slot)` 间接访问。但这个代价相对于专家 FFN 的 int8 GEMM 通常较小，尤其在 S3 中每个 bucket 足够大时，AMX 带来的收益远大于分桶开销。S4 中 bucket 较稀疏，分桶收益更依赖 AMX 的高吞吐和任务数量；这也是为什么调度策略需要保留可实验空间。

`slot` 的存在避免了另一个潜在问题：同一个 token 有多个 routed expert，如果只按 expert 记录 token id，计算结束后无法知道输出应该落在该 token 的第几个 routed 缓冲位置。用 `base = token * MAX_TOP_K + slot` 后，每个 routed 输出拥有唯一连续空间，最终合并时只需按 token 顺序扫描 `top_k` 个 slot，合并阶段又重新变成连续向量加法。这个可以很好地协调负责不同 expert 的线程在处理同一个 token 时的 out 写冲突问题。

分桶后的访问模式可以理解为一“计算顺序”和“输出顺序”的解耦。计算时按 expert 顺序组织，最大化权重复用和 AMX tile 利用率；输出时再按 token 顺序合并，最大化最终写回的连续性。`bucket_token` 和 `bucket_slot` 就是连接这两种顺序的轻量索引。由于这些数组只保存整数，规模远小于专家权重和中间激活，额外带宽开销可以接受。

4.7 S4 batch router AMX 优化

S4 中 router 是大头之一。当前固定为 `4 experts x 4 tokens`：

```
1 for e in expert groups of 4:
2     for t in token groups of 4:
3         for cb in D / 16:
4             load 4 weight vectors
5             load 4 token vectors
6             do 16 FMA accumulators
7         reduce 16 dots
8         vector sigmoid
9         store 4 x 4 scores
```

循环展开后，核心结构为：

```

1 w0, w1, w2, w3 = router weights for experts e ... e+3
2 x0, x1, x2, x3 = tokens t ... t+3
3
4 acc00 += w0 * x0, acc01 += w1 * x0, ...
5 acc10 += w0 * x1, acc11 += w1 * x1, ...
6 ...

```

代码中的核心语句就是对 4 个 token 复用同一组权重：

```

1 acc00 = _mm512_fmadd_ps( w0, x0, acc00 );      /* token0 x expert0 */
2 acc01 = _mm512_fmadd_ps( w1, x0, acc01 );      /* token0 x expert1 */
3 acc10 = _mm512_fmadd_ps( w0, x1, acc10 );      /* token1 x expert0 */
4 acc11 = _mm512_fmadd_ps( w1, x1, acc11 );      /* token1 x expert1 */
5 ...

```

为什么是选择 4x4?

- 4 experts 与 router packing 的 row block 一致
- 4 tokens 让权重 load 复用 4 次
- 16 个 accumulator 加上 4 个 weight、若干 token register，仍能放进 AVX-512 的 32 个 ZMM 寄存器，不溢出
- 若 T 更大，寄存器压力上升，可能产生 spill；若 T 更小，权重复用不足。

最后写回 s_out 时，每个 token 只写 4 个 expert score，因此显式使用 128-bit store：

```

1 _mm_storeu_ps( st0, _mm_load_ps( dots ) );      /* token0 的 4 个 expert
score */
2 _mm_storeu_ps( st1, _mm_load_ps( dots + 4 ) );  /* token1 的 4 个 expert
score */
3 _mm_storeu_ps( st2, _mm_load_ps( dots + 8 ) );  /* token2 的 4 个 expert
score */
4 _mm_storeu_ps( st3, _mm_load_ps( dots + 12 ) ); /* token3 的 4 个 expert
score */

```

这个 router kernel 的核心是“权重 load 复用”。在一个 cb 上，4 个 expert 的权重向量 $w_0..w_3$ 只加载一次，然后分别与 4 个 token 的向量相乘，产生 16 个 dot accumulator。如果使用传统的 token 外层循环，每个 token 都会重新加载同样的 router 权重，权重带宽压力更大。4x4 tile 把一部分权重带宽转化为寄存器复用。展开 4x4 循环还有一个微架构层面的好处：编译器可以看到所有 accumulator 的独立性，生

成更直接的 FMA 指令序列。若保留二重小循环，编译器虽然可能展开，但会受到数组索引和别名分析影响。手工展开后，`acc00 ... acc33` 是明确的独立变量，利于寄存器分配和指令调度。

这个 kernel 写死为 $R = 4, T = 4$ 。 $T = 4$ 已经让每次权重 load 被 4 个 token 复用；继续增大 T 会线性增加 accumulator 数量，例如 4×8 需要 32 个 accumulator，几乎占满全部 ZMM，临时寄存器没有空间，容易 spill。减小 T 则降低权重复用，router 又会更偏向 memory-bound。 4×4 因此是一个比较均衡的点：寄存器还够，权重复用明显，循环体也足够大。

4.8 S4 select_topk_2_simd 向量化优化

S4 固定 $K = 2$ ，因此可以使用专门的 top-2 SIMD。这里可以优化的原因是 S4 需要从 512 个 experts 中选择 2 个，形状很极端（相较于 S1~S3 的 $(16, 4)$ ）。针对其中的 512 可以做较为明显的向量化优化。

它把 experts 按 16 个一组扫描：

```
1 lane 0: expert 0, 16, 32, ...           /* 独立维护 top1 和 top2 */
2 lane 1: expert 1, 17, 33, ...           /* 独立维护 top1 和 top2 */
3 ...
4 lane15: expert 15, 31, 47, ...          /* 独立维护 top1 和 top2 */
```

每条 lane 保存局部 top1/top2：

```
1 max1, idx1                               /* 当前 lane 的第一大值和 expert id */
2 max2, idx2                               /* 当前 lane 的第二大值和 expert id */
```

对当前 chunk：

```
1 v = s[ e : e + 15 ] + bias[ e : e + 15 ] /* Top-K 比较使用 biased score */
2 idx_new = e + lane                       /* 当前 16 个 expert 的 id */
```

更新 top1：

```

1  m1 = ( v > old1 )           /* 新值是否超过 top1 掩码 */
2  max1 = max( old1, v )       /* 更新 top1 value */
3  idx1 = m1 ? idx_new : idx1  /* 同步更新 top1 index */

```

更新 top2 的关键是：

$$candidate_2 = \min(old1, v). \quad (60)$$

因为：

- 若 $v > old1$ ，新值成为 top1，旧 top1 是 top2 候选；
- 若 $v \leq old1$ ，旧 top1 保持，新值是 top2 候选。

所以：

```

1  lo = min( old1, v );        /* top2 候选: 旧 top1 或当前值 */
2  m2 = ( lo > max2 );         /* 是否超过已有 top2 */
3  max2 = m2 ? lo : max2;      /* 更新 top2 value */
4  idx2 = m2 ? idx_lo : idx2;  /* 更新 top2 index */

```

所有 chunks 结束后，16 条 lane 各有 top1/top2，共 32 个候选。最后用标量 reduce 得到全局 top2。相等时使用严格大于更新，保留较小 index，符合 reference 的 tie-break。

这个算法比通用 Top-K 更适合 S4 的原因在于它只需要一遍扫描。通用 Top-K 往往做 K 轮 max scan，每选中一个 expert 就把它屏蔽掉；当 $K=2$ 时，这意味着至少两次遍历 512 个 expert。

select_topk_2_simd 在同一遍扫描中同时维护第一名和第二名，访存量近似减半。由于 s 和 bias 都是连续数组，按 16 expert chunk 读取可以稳定使用 512-bit load。最后 32 个候选使用标量归并而不是继续写复杂 SIMD reduce，是因为候选数量已经很小，标量代码更简单，也更容易精确处理 tie-break。这个设计体现了一个常见原则：SIMD 用在大规模规则扫描上，小规模控制逻辑则保留标量，避免为了完全向量化引入过多 shuffle 和 mask 操作。

Top-2 还必须注意 bias 与 gate 的语义区别。选择 expert 时比较的是 $score + bias$ ，但归一化 gate 时使用的是原始 router score。也就是说，bias 只参与路由决策，不应该改变实际 gate 权重。如果在 SIMD top-2 中把 bias 后的值继续用于 gate_sum，就会和 reference 语义不一致。实现中将“选择用值”和“归一化用值”分开，才能保持正确性。

4.9 最终累加向量化优化

shared expert 先写入 $y = x + \text{shared_out}$, routed expert 阶段把每个 (token, slot) 的输出写入 g_routed_out 。最后一步只需要把同一个 token 的若干 routed 输出加回 y 。这个阶段没有复杂数学过程, 而且访问规则、无分支、无跨 token 写冲突, 因此适合显式 AVX-512 向量化:

```
1  acc = load( y );                /* 已包含 x + shared_out */
2  acc += load( routed_out0 );     /* 加第一个 routed expert */
3  acc += load( routed_out1 );     /* S4 的第二个 routed expert */
4  store( y, acc );                /* 写回最终 token 输出 */
```

S4 中 top_k = 2 , 因此可以把两个 routed 输出的累加写成固定路径; S3 中 top_k = 4 , 也可以按 16 个 FP32 一组累加。这样做的收益不是改变算法复杂度, 而是避免标量循环中的逐元素 load/add/store。每个 ZMM 一次处理 16 个输出维度, $D = 256/512$ 都是 16 的倍数, 循环不需要 tail mask, 指令流非常规则。所以显式写成 AVX-512 是有收益的。

从并行角度看, 最终累加按 token 切分最自然。每个线程只写自己负责 token 的 $y[t]$, 不同 token 的内存区间互不重叠, 不需要 atomic 或 reduction. 由于每个 token 的工作量几乎完全相同, 这一段使用 schedule(static) 更合适: 静态调度开销低, 且连续 token 更容易保持较好的 cache locality.

5. 并行流与硬件利用分析

5.1 数据级并行

AVX-512 的一个 ZMM 寄存器可容纳 16 个 fp32 或 64 个 int8。代码中的常见块大小都围绕这个容量设计:

- router: 16 FP32 dims;
- quantize/requantize: 16 FP32 -> 16 INT8;
- VNNI gate/up: 64 INT8 input block;
- down packed: 32/64 INT8 hidden block;
- final accumulate: 16 FP32 outputs。

这样大部分热循环都没有尾部处理, 因为 driver 保证 D 和 H 是 64 的倍数。没有 tail mask 的好处不仅是少几条指令, 也避免了 mask register 参与数据路径, 使编译器更容易生成简单循环。对高频内核而言, 循环体越规则, 前端解码、分支预测和后端调度越稳定。

5.2 线程级并行

不同 case 的线程级并行来源不同：

Case	主要 TLP	原因
S1	expert workers	N=1，只能并行 shared/routed experts
S2	expert workers + projection helper	单专家矩阵大，专家内矩阵乘法也可以拆
S3	token preprocessing + AMX bucket tasks	N=128，E=16，bucket 较满
S4	expert-group router + bucket tasks	E=512，router/top-k 和稀疏分桶

线程级并行的核心不是“线程越多越好”，而是任务粒度要足够大且负载要尽量均衡。S1/S2 的任务数量天然较少，因此用 persistent worker 控制固定数量的专家/矩阵任务；S3/S4 的任务数量来自 token block 和 expert bucket，更适合 OpenMP 管理。对于 routed AMX task，bucket 大小受 router 输出影响，存在不均匀性，所以动态调度通常更稳；而最终累加这种每 token 工作量固定的阶段，则使用 static 更合适。

还需要区分同步的两类成本：线程调度成本和阶段 barrier 成本。调度成本来自任务分配太碎，barrier 成本来自某些线程落后导致其他线程等待。S3/S4 中 `schedule( dynamic, 1 )` 增加了一点调度开销，但能减少路由不均衡带来的长尾等待；最终累加用 static 则是因为不存在长尾，动态调度只会徒增开销。

5.3 计算密度

单 token matrix-vector 的权重复用很差：一块权重只服务一个 token. batch 后按 expert 分桶，变成 matrix-matrix 矩阵乘法：

```
1 16 tokens x 64 K  times  64 K x 16 rows
```

此时一个权重 tile 服务 16 token，一个 activation tile 服务 16 output rows，算术强度显著提高。这是 S3/S4 使用 AMX 的根本原因。可以粗略比较两种组织方式的复用：matrix-vector 中，一段 expert 权重被一个 token 消费后就结束；AMX tile 中，同一段权重同时参与 16 个 token 的输出。反过来，同一个 activation tile 也被 16 个输出行共享。这样单位内存读取对应的 MAC 数增加，程序更容易从 memory-bound 向 compute-bound 靠近。

5.4 流水化与寄存器压力

AVX-512 router 4x4 kernel 展开后有 16 个 independent accumulators。这样 FMA 之间没有长依赖链，乱序执行器可以在多个 accumulator 间交错发射，隐藏 FMA latency。代价是寄存器压力上升；Sapphire Rapids 有 32 个 ZMM，4x4 是一个饱和点。AMX 部分也尽量使用多个 independent tile accumulators。例如 shared gate/up 同时维护 gate 和 up 两个 tile accumulator，让 TMUL 和 load/store 有更多可重叠空间。

寄存器压力是展开优化的主要边界。展开能增加独立 accumulator，帮助隐藏延迟；但展开过度会导致寄存器 spill，把原本寄存器内的临时值写到栈上，立刻损失性能。因此报告中多次出现的 4 rows、4 tokens、16 outputs 都不是任意的，而是在硬件寄存器数量、向量宽度、权重复用和循环体复杂度之间取得的折中，在不同配置下测试后的最优方案。优化的目标不是把循环全部展开，而是展开到刚好能提供足够指令级并行。

6. 四个 case 的最终加速效果

不同的 case 的单次 forwarding 时间长短不一，故采用不同的迭代次数。

S1 case:

```
root at h3240105662-pod1 in ~/hpc101/src/lab2 on mainxxx 26-08-05 - 13:50:40
.: hpc submit -p lab2 -c 16 ./build/lab2 1 256 128 16 4 100000
submitted job 42227
problem size: num_tokens=1 d_model=256 d_ff=128 num_experts=16 top_k=4 (100000 iterations)
Baseline time: 5.14844 s
Optimized time: 0.281553 s
Result is correct! (rel RMSE: 0.0019397, worst token: 0.0019397 at 0)
Speedup: 18.2859
```

S2 case:

```
[root at h3240105662-pod1 in ~/hpc101/src/lab2 on mainxxx 26-08-05 - 13:52:52
.: hpc submit -p lab2 -c 16 ./build/lab2 1 1024 512 16 4 10000
submitted job 42243
problem size: num_tokens=1 d_model=1024 d_ff=512 num_experts=16 top_k=4 (10000 iterations)
Baseline time: 6.25948 s
Optimized time: 0.424603 s
Result is correct! (rel RMSE: 4.30069e-08, worst token: 4.30069e-08 at 0)
Speedup: 14.742
```

S3 case:

```
[root at h3240105662-pod1 in ~/hpc101/src/lab2 on mainxxx 26-08-05 - 13:54:22
.: hpc submit -p lab2 -c 16 ./build/lab2 128 256 128 16 4 10000
submitted job 42247
problem size: num_tokens=128 d_model=256 d_ff=128 num_experts=16 top_k=4 (10000 iterations)
Baseline time: 65.9322 s
Optimized time: 0.675547 s
Result is correct! (rel RMSE: 0.00114206, worst token: 0.00334846 at 13)
Speedup: 97.5983
```

S4 case:

```
[root at h3240105662-pod1 in ~/hpc101/src/lab2 on mainxxx 26-08-06 - 3:49:03
.: hpc submit -p lab2 -c 16 ./build/lab2 1024 512 128 512 2 100
submitted job 44065
problem size: num_tokens=1024 d_model=512 d_ff=128 num_experts=512 top_k=2 (100 iterations)
Baseline time: 22.9423 s
Optimized time: 0.179904 s
Result is correct! (rel RMSE: 0.000663915, worst token: 0.00913755 at 499)
Speedup: 127.525
```

7. 思考题

7.1 估计访存强度

以场景 S3 ($N = 128$, $D = 256$, $H = 128$, $E = 16$, $K = 4$) 为例, 估算参考实现的一次前向的总访存量 (专家权重被读了多少遍?) 和总的乘加次数, 计算算数强度 (MACs/byte)。按专家分组之后这两个数字分别变为多少? 由此说明这个负载是访存瓶颈还是计算瓶颈, 以及分组为什么可以加速。

对于一次前向而言, 主要的计算量是 W_{gate} , W_{up} 和 W_{down} 与 token 之间的 INT8 类型的矩阵乘法。此外还有 router 的 FP32 类型乘法、两次量化等等杂项计算。我们主要考虑规模较大的矩阵乘法, 这里 W_{gate} , W_{up} 以及 W_{down} 的矩阵规模都是 $DH = 2^{15}$ 。

7.1.1 参考实现

参考实现是 token-centric 的计算形式，对于每一个 token，加载 $(K + 1)$ 个 experts 的权重参数来做矩阵计算。因此，除共享专家外，平均每个专家被读取 $\frac{N \times K}{E} = \frac{128 \times 4}{16} = 32$ 次，则总的访存量约为（单个权重为一字节的 INT8）：

$$N \times (K + 1) \times D \times H = 2^{7+8+7} \times 5 \approx 6.29 \times 10^7 \text{ bytes} \quad (61)$$

在 token-centric 的场景下，专家权重基本上就是 load 一个字节，做一次乘加运算，所以总的乘加次数为：

$$N \times (K + 1) \times D \times H = 2^{7+8+7} \times 5 \approx 6.29 \times 10^7 \text{ MACs} \quad (62)$$

因此，算数强度基本上就是处于 1 MACs/byte 的水平，这个算数强度很低，属于访存瓶颈。

7.1.2 按专家分组后

按专家分组后，在理想情况下，128 个 tokens 的总计 $128 \times 4 = 512$ 个 routes 能够均匀地分担到 16 个专家处，因此每一个专家平均被 $\frac{N \times K}{E} = \frac{128 \times 4}{16} = 32$ tokens 使用。通过 AMX 矩阵加速时，每一次 16 个 tokens 可以共享同一份专家权重。因此，每一份专家权重平均被读取 2 次，总的访存量：

$$\frac{N}{16} \times (K + 1) \times D \times H = 2^{3+8+7} \times 5 \approx 3.93 \times 10^6 \text{ bytes} \quad (63)$$

但是总的乘加次数不变，毕竟矩阵计算不可省：

$$N \times (K + 1) \times D \times H = 2^{7+8+7} \times 5 \approx 6.29 \times 10^7 \text{ MACs} \quad (64)$$

因此，算数强度基本上翻到参考实现的 16 倍，即 16 MACs/byte，接近计算瓶颈。

所以，通过分组，在理想情况下，同一份 expert 权重被访问后可以被 16 个 tokens 共享复用。所以等量的访存之后可以做约 16 倍乘加计算，计算密度大大增加，从访存瓶颈切换为计算瓶颈。

7.2 关于激活值

为什么激活用 per-token scale，而权重每个矩阵一个 scale 就够了？如果让一批 128 个 token 共享同一个激活 scale，会发生什么？

其实，使用 per-token scale 还是 batch-token scale 本身在逻辑上都没有问题。关键就是实际的数值精度会存在差异，我们将 FP32 量化为 INT8 的目的是通过降低精度提高计算效率，但是不应该过分牺牲精度。在 MoE 输入的 token 中，不同的 token 的数值特性不一样，比如 token1 的最大分量是 100，而 token2 的最大分量是 10。这是如果采用 batch-token scale，统一将全部 token 范围的最大分量作为 s_x ，则 token2 这类 token 就会被压缩到 0 附近的 INT8 的值，这样极有可能在计算时塌缩为 0 造成计算误差放大。

因此，batch-token 整体做 scale 可能会造成极大误差。本身 token 与 token 之间就是独立的、动态变化的，直觉上也是应该分开做量化，分开取 scale。而对于一个权重矩阵而言，矩阵整体是稳定的（在模型训练后就保持不变），可以共用一个 scale。换句话说，权重矩阵在内存中的排布本身就是一维的，可以看成一个很长的 token，只是逻辑上是二维的而已，应当作为一个 token 来处理。

7.3 关于数值范围

单个 int8 * int8 乘积的最大绝对值是多少？为什么点积必须在 int32 中累加 —— 若改用 int16，最坏情况下累加到第几项就会溢出？规约长度在运行时变化，请用允许的最大规约长度（MAX_D_MODEL = 1024）估算最坏情况下的累加值，并说明它离 int32 的表示上限还有多少余量？

单个 int8 的数值范围是 $[-128, 127]$ ，则 int8 * int8 乘积的最大绝对值是 $(-128) \times (-128) = 2^{14} = 16384$ 。而 int16 的数值范围是 $[-32768, 32767]$ ，则 int8 * int8 最大值的 2 倍（ $16384 \times 2 = 32768$ ）就超过了 int16 的表示范围。因此，必须要用 int32 作为累加器的类型。如果采用最大规约长度 1024，则最坏情况下的累加值为：

$$1024 \times 128 \times 128 = 2^{24} \tag{65}$$

而对于 int32 类型的表示范围 $[-2^{31}, 2^{31} - 1]$ 而言，上述最坏情况下的累加值只是占用：

$$\frac{2^{24}}{2^{31} - 1} \approx \frac{1}{2^7} \approx 0.78\% \tag{66}$$

此时，距离 int32 的表示上限还有 $(128 - 1) \times 1024 \times 128 \times 128 = 2.13 \times 10^9$ 。

Lab2 实验报告的全部内容如上。